Read the Instructions that I have added

Yes — I've read and understood the instructions you added in the "Booking chatbot" Space.

To recap what you're asking for in this project:

- You want an **autonomous AI Agent** that:
  - Checks your **Google Calendar** for demo availability (via a Google Service Account).
  - Creates a calendar **event** when a demo is booked.
  - Logs the **lead** into a database (PostgreSQL/Supabase).
  - Sends a **confirmation email** via **Resend**.
- You want concrete guidance on:
  1. **Google Service Account setup** (Cloud Console checklist, sharing your calendar with the service-account email, and granting write access).
  2. **Agentic workflow and tool logic** using the **OpenAI Agents SDK** for:
     - `get_available_slots` (querying freebusy over the next 5 business days),
     - `create_booking` (create event + write to DB),
     - `send_summary_email` (trigger Resend after success).
  3. **Timezone handling** best practices between:
     - Next.js (user's local time),
     - Python backend (UTC),
     - Google Calendar API (proper syncing).
  4. A **Python code blueprint** using the OpenAI Agents SDK that includes a **validation loop** where the AI confirms the date/time with the user before executing `create_booking`.
  5. A **simple SQL schema** for a `leads` or `bookings` table, including:
     - name, email, company,
     - and the **Google Event ID** so you can later reschedule or cancel.

I'll now start drafting a **step-by-step, implementation-ready** answer tailored to your stack (Next.js, FastAPI, OpenAI Agents SDK, Resend, PostgreSQL, Google Calendar API).

Would you like me to begin with #1: Google Service Account setup checklist, or jump straight into the **agentic tool logic + Python code blueprint**?

What library should I use to create chatbot UI /

For your stack (Next.js + FastAPI + OpenAI Agents), you have two good options:

1. Use an existing "AI chatbot UI" kit

For a **quick, polished UI**, consider:

- **Vercel's AI Chatbot template** (Next.js + AI SDK)
  - Uses `@vercel/ai` (`useChat` hook) and gives you a full streaming chat UI out of the box.[10] [11]
  - Works well with your Next.js App Router and plays nicely with the OpenAI Agents SDK on the backend.
- **chatscope/chat-ui-kit-react**
  - Lightweight React chat-UI components (message bubbles, input bar, etc.).[12]
  - You control the backend logic and just plug in your chat messages.
- **react-chatbotify**
  - Modern React library for building flexible chatbots with custom flows and styling.[13]
  - Good if you want to add step-by-step booking flows (e.g., "pick date → confirm → book").

2. Build your own minimal chat UI with a component library

If you prefer **full control** and consistency across your app:

- Use **shadcn/ui** (Tailwind-based) for buttons, inputs, dialogs, etc.[14]
- Pair it with a **simple chat component** you write yourself (message list + `textarea` + submit button).
- This is very clean when you already have FastAPI + OpenAI Agents on the backend and just need a dumb UI layer.

Recommendation for your use case

Given your project is a **demo-booking chatbot**:

- Start with **Vercel's AI Chatbot template** or **AI SDK** `useChat` for the UI.[11] [10]
- Then replace the default backend logic with your **FastAPI OpenAI Agents SDK endpoint** that calls Google Calendar, DB, and Resend.

If you tell me whether you want something **minimal-custom** or **feature-rich-prebuilt**, I can sketch the exact `Chat` component file (React + Next.js) tailored to your booking flow.

⁂

Basic Structure for, how to integrate with nextjs

Here's a **minimal, clean structure** to integrate your booking-chatbot UI with Next.js (App Router) and your FastAPI backend:

1. Project layout (Next.js side)

With the App Router, keep it simple:

```
app/
  chat/          → your chatbot page
    page.tsx     → the main chat UI
  api/           → (optional) Next.js API routes if you don't proxy
components/
  Chat.tsx       → reusable chat UI
  Message.tsx    → single message bubble
  InputBar.tsx   → user input + send button
```

You can also put the chat component directly in `app/chat/page.tsx` if you don't need re-use.

2. Basic chat UI component

A simple `Chat.tsx` using `fetch` to your FastAPI endpoint:

```
// components/Chat.tsx
"use client";

import { useState, useEffect, useRef } from "react";

type Message = {
  role: "user" | "assistant";
  content: string;
};

export default function Chat() {
  const [messages, setMessages] = useState<Message[]>([]);
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false);
  const messagesEndRef = useRef<HTMLDivElement>(null);

  // Scroll to bottom on new messages
  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!input.trim() || loading) return;

    const userMessage: Message = { role: "user", content: input };
    setMessages((prev) => [...prev, userMessage]);
```

```
    setInput("");
    setLoading(true);

    try {
      const res = await fetch("/api/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ messages: [...messages, userMessage] }),
      });

      const { response } = await res.json();
      setMessages((prev) => [...prev, { role: "assistant", content: response }]);
    } catch (err) {
      setMessages((prev) => [
        ...prev,
        { role: "assistant", content: "Sorry, something went wrong." },
      ]);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="flex flex-col h-screen max-w-2xl mx-auto">
      <div className="flex-1 overflow-y-auto p-4 space-y-4">
        {messages.map((m, i) => (
          <div
            key={i}
            className={`flex ${m.role === "user" ? "justify-end" : "justify-start"}`}
          >
            <div
              className={`max-w-xs rounded-lg px-4 py-2 ${
                m.role === "user"
                  ? "bg-blue-600 text-white"
                  : "bg-gray-200 text-gray-800"
              }`}
            >
              {m.content}
            </div>
          </div>
        ))}
        <div ref={messagesEndRef} />
      </div>

      <form
        onSubmit={handleSubmit}
        className="flex gap-2 p-4 border-t"
      >
        <input
          type="text"
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="Type your message..."
          className="flex-1 border rounded-md px-4 py-2"
          disabled={loading}
        />
```

```
        <button
          type="submit"
          disabled={loading}
          className="bg-blue-600 text-white px-4 py-2 rounded-md"
        >
          {loading ? "…" : "Send"}
        </button>
      </form>
    </div>
  );
}
```

(You'd then plug this into `app/chat/page.tsx`.)

3. How it connects to FastAPI

On the Next.js side, you typically:

- Either:
  - Call FastAPI directly from the client (e.g., `await fetch("http://localhost:8000/chat")`), or
  - Use a **Next.js API route** as a proxy:

```
// app/api/chat/route.ts
// Next.js route handler (optional proxy)
export async function POST(request: Request) {
  const { messages } = await request.json();

  const res = await fetch("http://localhost:8000/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ messages }),
  });

  const data = await res.json();
  return Response.json(data);
}
```

This keeps CORS and auth logic contained on the backend.

4. If you use Vercel AI SDK (optional)

If you want to lean into the AI-SDK pattern:

- Install `@ai-sdk/react` and create a route handler in `app/api/chat/route.ts` wrapping your FastAPI-backed agent.[20] [21]
- Then replace the `fetch` in `Chat.tsx` with `useChat` from `@ai-sdk/react`.[22] [21]

If you want, I can next generate:

- A full `app/chat/page.tsx` wired to your FastAPI agent, and
- A matching FastAPI `/chat` endpoint that talks to the OpenAI Agents SDK and Google Calendar.

⁂

1. https://www.bbc.co.uk/bitesize/articles/z9r3n9q
2. https://www.reddit.com/r/Professors/comments/zmzj6y/adding_a_did_you_read_the_instructions_component/
3. https://www.linkedin.com/pulse/read-instructions-first-youre-welcome-scott-wance
4. https://www.reddit.com/r/Professors/comments/prhak4/how_do_i_get_them_to_read_instructions_no_really/
5. https://ludwig.guru/s/read+the+instructions+carefully
6. https://ludwig.guru/s/i+have+read+and+understand+the+instructions
7. https://www.facebook.com/e2language/posts/read-the-instructions-fully-and-carefully-before-attempting-to-answer-ielts-ielt/952915196875983/
8. https://www.youtube.com/watch?v=pmRhC_wb9G8
9. https://www.reddit.com/r/Teachers/comments/xft6a2/read_the_instructions/
10. https://vercel.com/templates/next.js/chatbot
11. https://ai-sdk.dev/docs/ai-sdk-ui/chatbot
12. https://github.com/chatscope/chat-ui-kit-react
13. https://github.com/react-chatbotify/react-chatbotify
14. https://www.reddit.com/r/nextjs/comments/1j8khc1/what_is_the_best_ui_component_library_for/
15. https://codesandbox.io/p/github/baronrustamov/chatbot-ui
16. https://github.com/dulanmalinda/ai-chatbot-ui
17. https://www.youtube.com/watch?v=4je2_HMCXU0
18. https://dev.to/salmenus/5-react-js-libraries-to-integrate-chatgpt-into-your-web-app-24e7
19. https://github.com/assistant-ui/assistant-ui
20. https://ai-sdk.dev/docs/getting-started/nextjs-app-router
21. https://blog.saeloun.com/2023/07/13/building-chatbot-in-next-js-using-vercel-ai-sdk/
22. https://www.telerik.com/blogs/practical-guide-using-vercel-ai-sdk-next-js-applications
23. https://www.freecodecamp.org/news/build-a-custom-ai-chat-application-with-nextjs/
24. https://customgpt.ai/embed-a-chatbot-nextjs-app/
25. https://ccgit.crown.edu/cyber-reels/next-js-and-fastapi-your-ultimate-starter-guide-1764801219
26. https://www.scribd.com/document/882830725/FastAPI-Backend-Setup-Integration-Guide
27. https://vercel.com/templates/next.js/multi-modal-chatbot
28. https://github.com/Kabi10/nextjs-ai-chatbot

29. https://pasly.co/blogs/connect-nextjs-to-fastapi